

## DAY 1

|                |                         |                        |
|----------------|-------------------------|------------------------|
| <b>Name:</b>   | <b>Abhishek Bankoti</b> | <b>UID: 23BCS10295</b> |
| <b>Branch:</b> | <b>BE-CSE</b>           | <b>Date:23/06/26</b>   |

### **Coding Problem 1: Interstellar Mining Fleet Manager**

```
abstract class SpaceVessel {
    short shipId;
    boolean operationalStatus;
    char fleetClassification;

    SpaceVessel(short shipId, boolean operationalStatus, char fleetClassification) {
        this.shipId = shipId;
        this.operationalStatus = operationalStatus;
        this.fleetClassification = fleetClassification;
    }
}

class MiningShip extends SpaceVessel {

    float[][] cargoHold;
    MiningShip(short shipId, boolean operationalStatus,
        char fleetClassification, float[][] cargoHold) {
        super(shipId, operationalStatus, fleetClassification);
        this.cargoHold = cargoHold;
    }
    public float calculateTotalOreWeight() {
        float total = 0;
        for (int i = 0; i < cargoHold.length; i++) {
            for (int j = 0; j < cargoHold[i].length; j++) {
                total += cargoHold[i][j];
            }
        }
        return total;
    }

    public float findHeaviestContainer() {
        float max = cargoHold[0][0];

        for (int i = 0; i < cargoHold.length; i++) {
            for (int j = 0; j < cargoHold[i].length; j++) {
                if (cargoHold[i][j] > max) {
                    max = cargoHold[i][j];
                }
            }
        }
        return max;
    }
}

public class Main {
    public static void main(String[] args) {
```

```

SpaceVessel[] fleet = new SpaceVessel[2];
float[][] cargo = {
    {1200.5f, 3400.2f},
    {4500.8f, 6000.1f}
};

fleet[0] = new MiningShip((short)100,
    true,
    'A',
    cargo);
MiningShip ship = (MiningShip) fleet[0];
System.out.println(ship.calculateTotalOreWeight());
System.out.println(ship.findHeaviestContainer());
}
}

```

#### Data Types Chosen

| Requirement               | Data Type |
|---------------------------|-----------|
| Ship ID (< 30000)         | short     |
| Bay Number (< 100)        | byte      |
| Operational Status        | boolean   |
| Fleet Classification      | char      |
| Ore Weight (50000.00)     | float     |
| Fleet Value (>40 Billion) | long      |

#### Coding Problem 2: Colony Grid Power Manager

```

class PowerManager {
    private byte sectorStates = 0;
    public void turnOnSector(int sectorIndex) {
        sectorStates = (byte)(sectorStates | (1 << sectorIndex));
    }
    public void turnOffSector(int sectorIndex) {
        sectorStates = (byte)(sectorStates & ~(1 << sectorIndex));
    }
    public boolean isSectorOn(int sectorIndex) {
        return (sectorStates & (1 << sectorIndex)) != 0;
    }
    public void display() {
        System.out.println(
            Integer.toBinaryString(
                sectorStates & 0xFF
            )
        );
    }
}

public class Main {
    public static void main(String[] args) {
        PowerManager pm = new PowerManager();

        pm.turnOnSector(0);
    }
}

```

```

        pm.turnOnSector(3);
        pm.display();
        System.out.println(pm.isSectorOn(3));
        pm.turnOffSector(3);
        pm.display();
    }
}

```

### Coding Problem 3: DNA Sequencer

```

class DNASequencer {
    private StringBuilder dnaSequence;
    DNASequencer(int expectedSize) {
        dnaSequence = new StringBuilder(expectedSize);
    }
    public void ingestSequence(char[] sensorData) {
        for (char c : sensorData) {
            dnaSequence.append(c);
        }
    }
    public void mutateDNA(String target,
                           String replacement) {
        int index = dnaSequence.indexOf(target);
        if (index != -1) {
            dnaSequence.replace(
                index,
                index + target.length(),
                replacement
            );
        }
    }
    public String getSequence() {
        return dnaSequence.toString();
    }
}

public class Main {
    public static void main(String[] args) {
        DNASequencer dna =
            new DNASequencer(100000);
        char[] data = {'A','C','T','G'};
        dna.ingestSequence(data);
        dna.mutateDNA("CT", "GG");
        System.out.println(
            dna.getSequence()
        );
    }
}

```

### Coding Problem 4: Deep-Sea Telemetry

```

import java.io.IOException;
class HardwareLockException extends Exception {
    public HardwareLockException(String msg) {

```

```

        super(msg);
    }
}
class SensorCorruptionException
    extends RuntimeException {
    public SensorCorruptionException(String msg) {
        super(msg);
    }
}
class TelemetryStream
    implements AutoCloseable {
    public void readData() {
        System.out.println("Reading telemetry...");
    }
    @Override
    public void close() {
        System.out.println("TelemetryStream Closed");
    }
}
public class Main {
    public static void parseTelemetry()
        throws HardwareLockException {
        try (TelemetryStream stream =
            new TelemetryStream()) {
            stream.readData();
            if (Math.random() > 0.5) {
                throw new HardwareLockException(
                    "File Locked"
                );
            }
            if (Math.random() > 0.5) {
                throw new SensorCorruptionException(
                    "Invalid Sensor Data"
                );
            }
        }
    }
    public static void main(String[] args) {
        try {
            parseTelemetry();
        }
        catch (HardwareLockException e) {
            System.out.println(e.getMessage());
        }
    }
}

```

### Coding Problem 5: Drone Hive Synchronization

```

import java.util.concurrent.atomic.AtomicInteger;
class DroneHive {
    private AtomicInteger totalDronesReturned =

```

```

        new AtomicInteger(0);
private volatile boolean emergencyAbort =
    false;
public void droneReturned() {
    totalDronesReturned.incrementAndGet();
}
public int getReturnedCount() {
    return totalDronesReturned.get();
}
public void triggerEmergencyAbort() {
    emergencyAbort = true;
}
public boolean isEmergencyAbort() {
    return emergencyAbort;
}
}
public class Main {
    public static void main(String[] args)
        throws InterruptedException {
        DroneHive hive = new DroneHive();
        Runnable drone = () -> {
            hive.droneReturned();
            if (hive.isEmergencyAbort()) {
                System.out.println(
                    "Drone rerouting..."
                );
            }
        };
        Thread[] drones = new Thread[10000];
        for (int i = 0; i < drones.length; i++) {
            drones[i] = new Thread(drone);
            drones[i].start();
        }
        hive.triggerEmergencyAbort();
        for (Thread t : drones) {
            t.join();
        }
        System.out.println(
            hive.getReturnedCount()
        );
    }
}

```